Algorithm 計算機演算法
Class 3: Dynamic Programming

逢甲大學 人工智慧研究中心
資 訊 工 程 學 系

許懷中

Dynamic Programming (動態規劃)

Dynamic Programming

- 將問題分成小塊
- 將小問題的結果先儲存起來
- 當需要小問題的計算結果以解決大問題時，使用先前儲存的結果，而非重新計算之
- 需要規劃計算小問題的順序

Dynamic Programming (cont.)

- 運用遞迴 (recurrence)，將大問題分解為規模較小但是性質相似的小問題
- **Bottom-up** 解題，從最小的問題開始，逐步向上求取最終問題的解
- 將小問題的解記錄下來，以備後續使用

Binomial Coefficient (二項式係數)

- $(a+b)^n = \sum_{k=0}^{n} \frac{n!}{k!(n-k)!} a^k b^{n-k}$
- $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ for $0 \le k \le n$
- 二項式係數的遞迴式
  - $\binom{n}{k} = \begin{cases} \binom{n-1}{k-1} + \binom{n-1}{k} & 0 < k < n \\ 1 & k = 0 \ or \ k = n \end{cases}$

二项式系数的性质

$(a+b)$						1		1					
$(a+b)^2$					1		2		1				
$(a+b)^3$				1		3		3		1			
$(a+b)^4$			1		4		6		4		1		
$(a+b)^5$		1		5		10		10		5		1	
$(a+b)^6$	1		6		15		20		15		6		1
…													
$(a+b)^n$	C_n^0		C_n^1		C_n^2				C_n^{n-2}		C_n^{n-1}		C_n^n

共有 $n+1$ 项

Divide-n-Conquer 解二項式係數

問題：計算二項式係數。

輸入：非負值的整數 n 及 k，其中 $k \le n$。

輸出：bin 二項式係數之值 $\binom{n}{k}$。

```
int bin (int n, int k)
{
  if (k == 0 || n == k)
      return 1;
  else
      return bin (n-1, k - 1)+ bin (n - 1, k);
}
```

這樣好嗎？

Divide-n-Conquer 解二項式係數 (cont.)

- Time complexity: $2\binom{n}{k} - 1$
- Why?

Dynamic Programming
解二項式係數

- 將$\binom{i}{j}$ $0 \le j \le i$的結果儲存於陣列$B[i][j]$
- $B[i][j] = \begin{cases} B[i-1][j-1] + B[i-1][j] & 0 < j < i \\ 1 & j = 0 \; or \; j = i \end{cases}$
- 從 i=o 解起

	0	1	2	3	4		j	k
0	1							
1	1	1						
2	1	2	1					
3	1	3	3	1				
4	1	4	6	4	1			
i								
n								

$B[i-1][j-1]$ $B[i-1][j]$

$B[i][j]$

範例

求 B[4][2] = $\binom{4}{2}$

計算第 0 列：　{ 這個步驟是為了完整地模仿演算法的定義。}

{ 雖然 $B[0][0]$ 這個值在稍後的計算中不會使用到。}

$$B[0][0] = 1$$

計算第 1 列：

$$B[1][0] = 1$$
$$B[1][1] = 1$$

計算第 2 列：

$$B[2][0] = 1$$
$$B[2][1] = B[1][0] + B[1][1] = 1 + 1 = 2$$
$$B[2][2] = 1$$

範例 (cont.)

計算第 3 列：

$$
\begin{aligned}
B[3][0] &= 1 \\
B[3][1] &= B[2][0] + B[2][1] = 1 + 2 = 3 \\
B[3][2] &= B[2][1] + B[2][2] = 2 + 1 = 3
\end{aligned}
$$

計算第 4 列：

$$
\begin{aligned}
B[4][0] &= 1 \\
B[4][1] &= B[3][0] + B[3][1] = 1 + 3 = 4 \\
B[4][2] &= B[3][1] + B[3][2] = 3 + 3 = 6
\end{aligned}
$$

Dynamic Programming 解二項式係數 (cont.)

問題：計算二項式係數。

輸入：非負值的整數 n 及 k，其中 $k \leq n$。

輸出：$bin2$，二項式係數之值 $\binom{n}{k}$。

```
int bin2 (int n, int k)
{
  index i, j;
  int B[0.. n] [0.. k];

  for (i = 0; i <= n; i++)
      for (j = 0; j <= minimum(i, k); j++)
         if (j == 0 || j == i)
            B[i] [j] = 1;
         else
            B[i] [j] = B[i -1][j -1] + B[i -1][j];
  return B[n][k];
}
```

Dynamic Programming 解二項式係數 (cont.)

i	0	1	2	3	…	k	$k+1$	…	n
迴圈執行次數	1	2	3	4	…	$k+1$	$k+1$	…	$k+1$

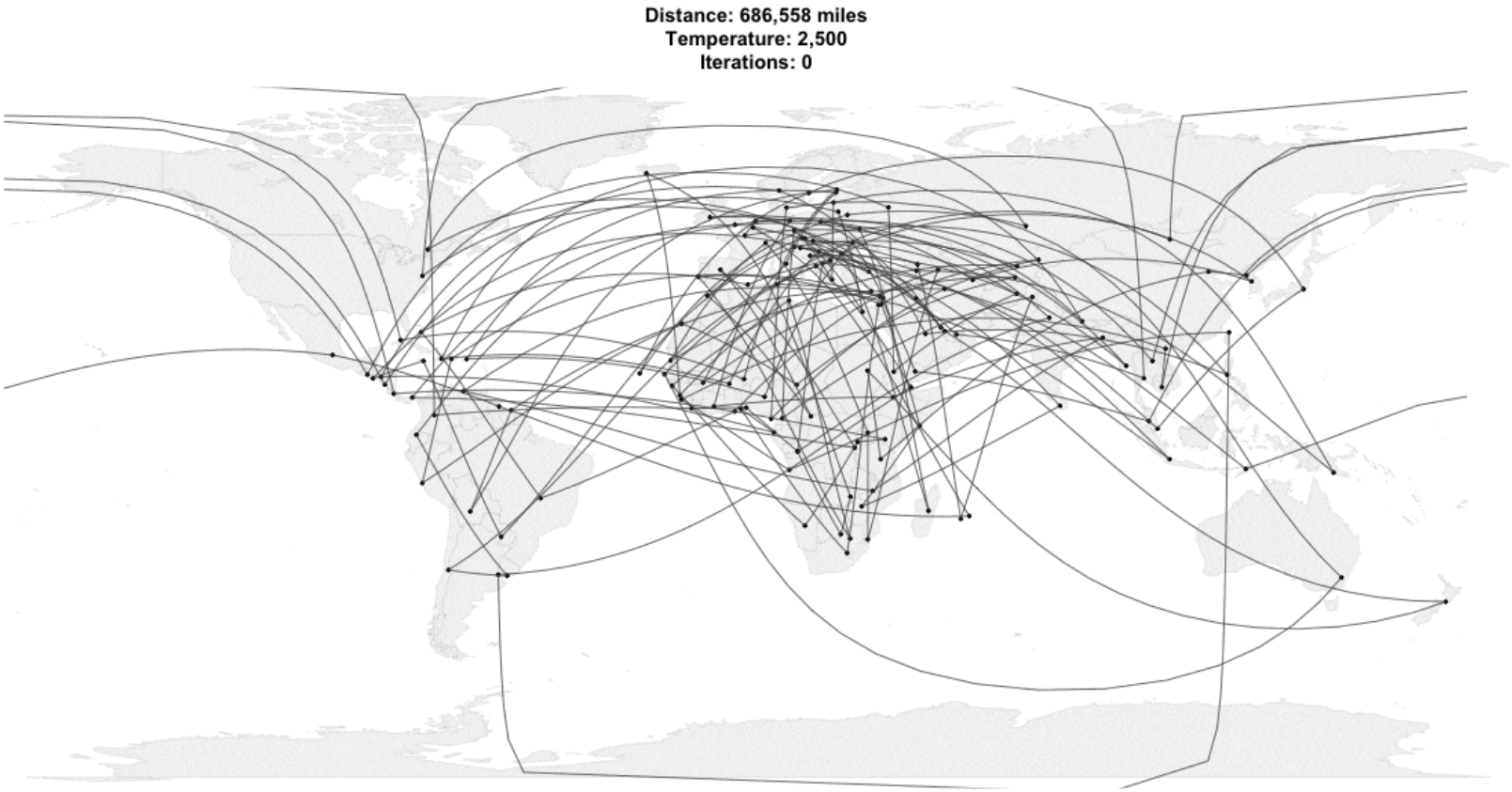
- 全部迴圈執行次數
  - $1+2+3+4+\cdots+k+\underbrace{(k+1)+(k+1)\cdots+(k+1)}_{n-k+1\text{個}}$
- $\frac{k(k+1)}{2}+(n-k+1)(k+1)=\frac{(2n-k+2)(k+1)}{2}\in\Theta(nk)$

Dynamic Programming 的應用

https://www.cs.utah.edu/~draperg/cartoons/dynamic.html

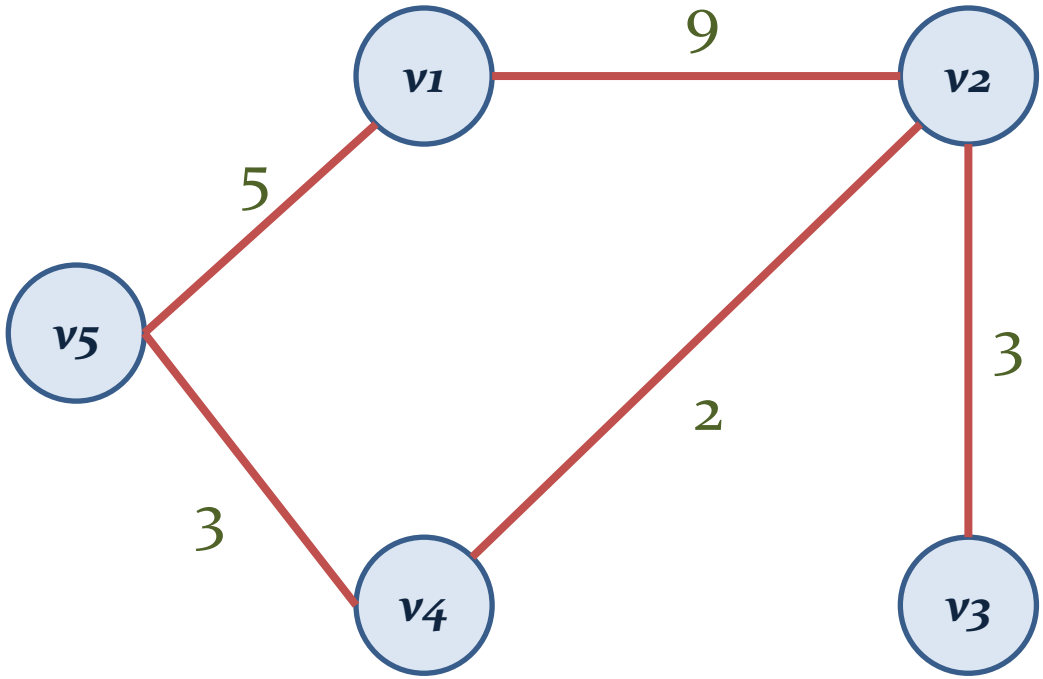
https://www.cs.utah.edu/~draperg/cartoons/dynamic.html

Shortest-Path Problem
(最短路徑問題)



Graph

Vertices



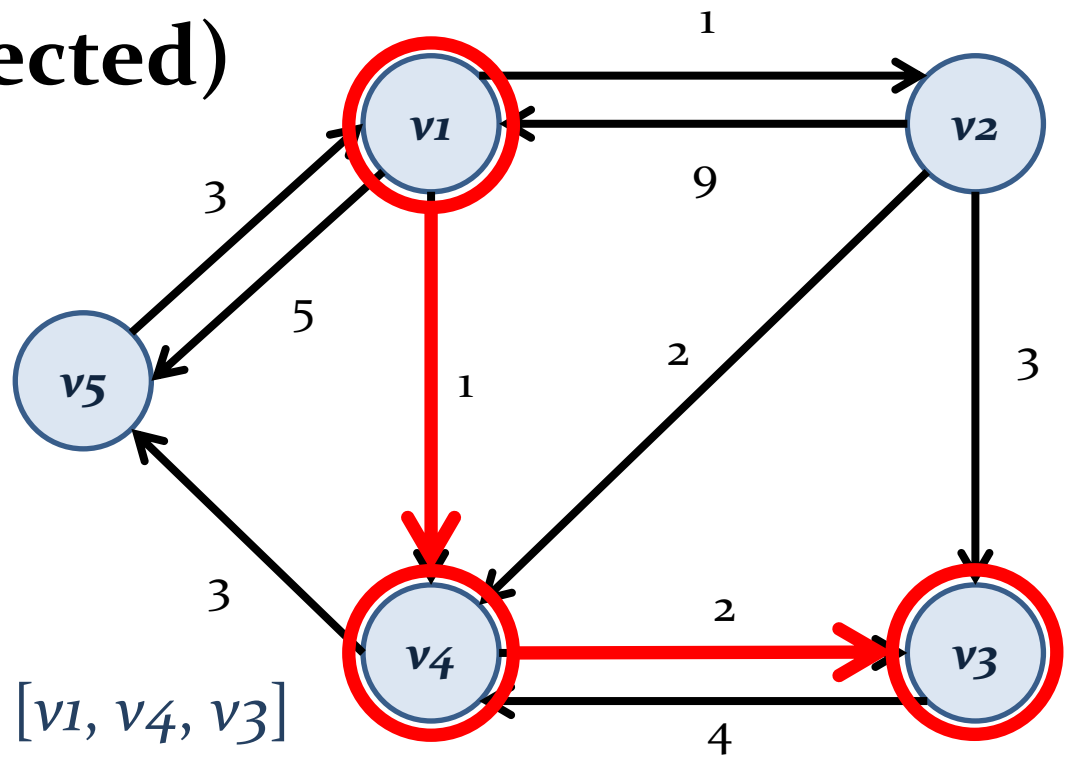
Edges (undirected)

Weights

Undirected Graph
Weighted Graph

Graph (cont.)

Edges (directed)



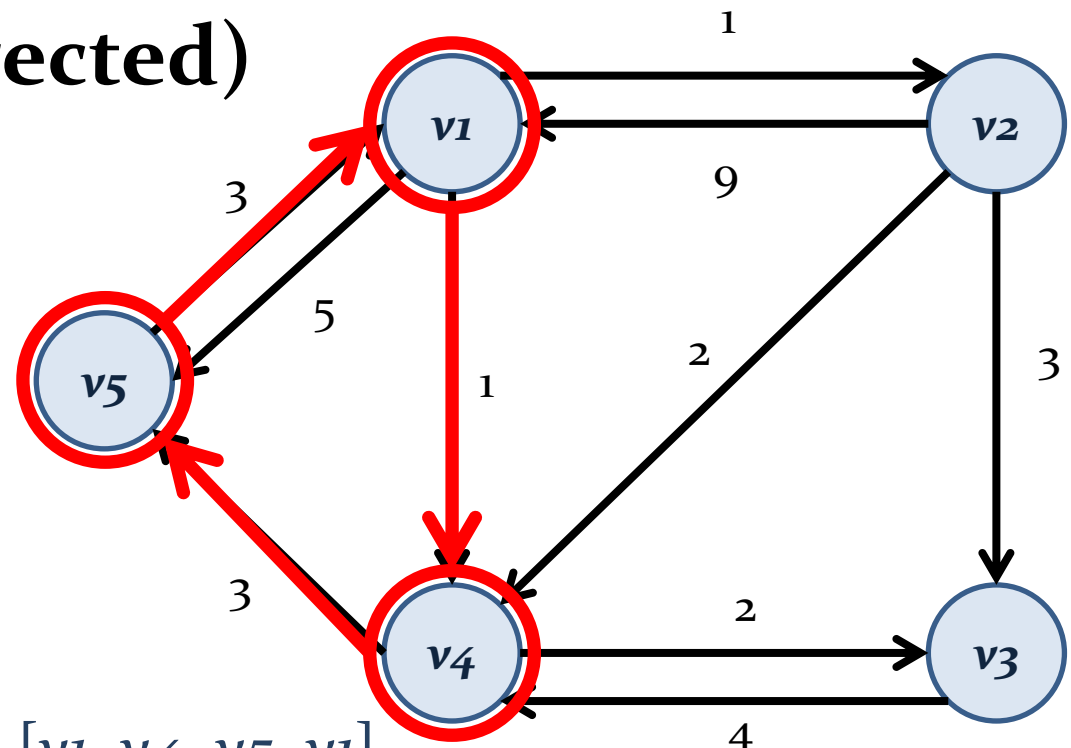
A path *v1* to *v3*: [*v1*, *v4*, *v3*]

The path [*v1*, *v4*, *v3*] is **simple**

The length of [*v1*, *v4*, *v3*] notated as length [*v1*, *v4*, *v3*] = **3**

Graph (cont.)

Edges (directed)



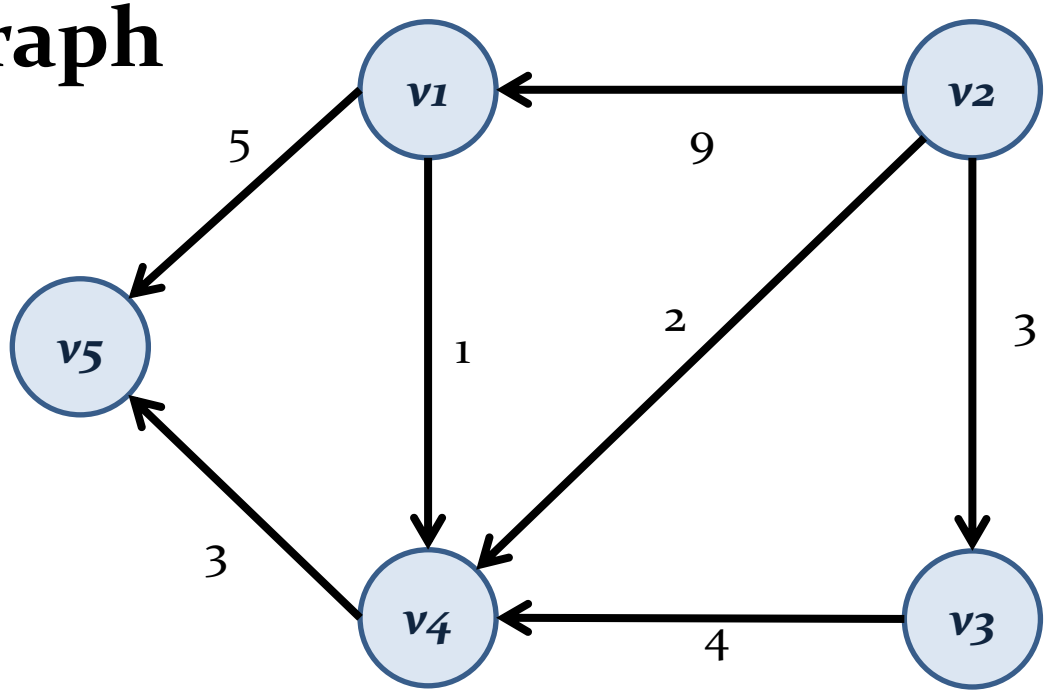
A path *v1* to *v1*: [*v1*, *v4*, *v5*, *v1*]

The path [*v1*, *v4*, *v5*, *v1*] is a **cycle**

Cyclic Graph

Graph (cont.)

Acyclic Graph



The Shortest Path Problem

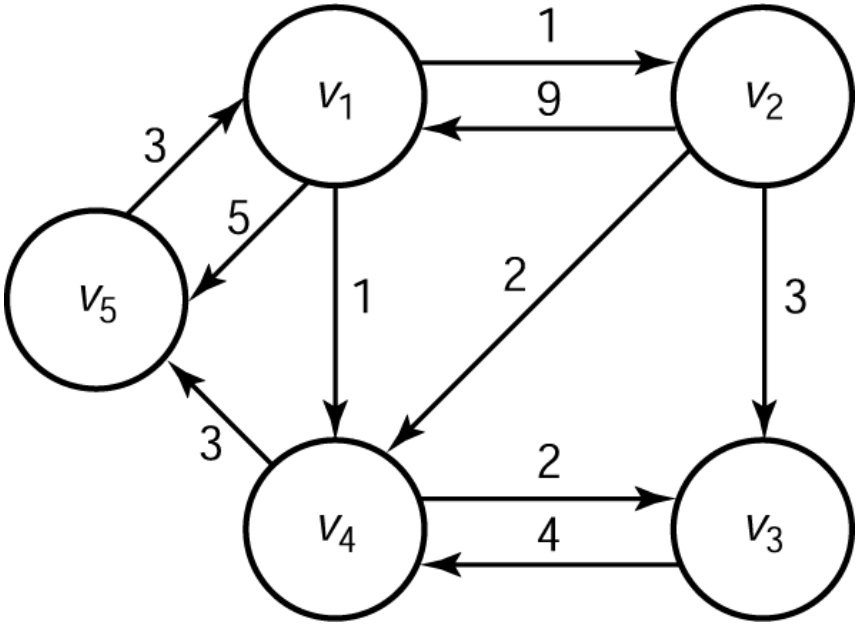
- 路徑搜尋問題 (Path Search)
  - 搜尋一條從某頂點到另一頂點的路徑
  - 可能存在多種可能
- 最短路徑問題
  - 一種最佳化的路徑搜尋問題
  - 所搜尋的路徑其長度須為最短 (minimum length)
- 最直覺的解法？

The Shortest Path Problem (cont.)

- 用暴力法搜尋最短路徑
  - 假設一個圖形有 n 個頂點，任一頂點皆有邊與其他頂點相連
  - 現在要尋找從 $v1$ 到 v_k 的最短路徑
  - Worst Case 要找幾次？
  - (n-2)! 超越指數時間的複雜度

The Shortest Path Problem (cont.)

- 若存在一由頂點 v_i 通往頂點 v_j 的邊，則稱 v_j 為 v_i 之鄰邊(v_j is adjacent to v_i)



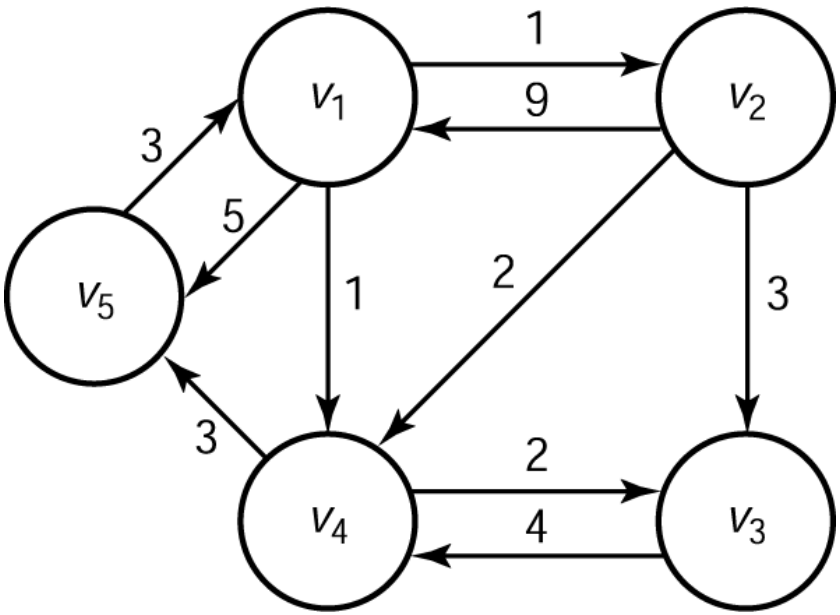
The Shortest Path Problem (cont.)

- 相鄰矩陣 W (Adjacency Matrix)
- $W[i][j] = \begin{cases} weights\ on\ edge & if\ \exists\ edge\ from\ v_i\ to\ v_j \\ \infty & if\ \nexists\ edge\ from\ v_i\ to\ v_j \\ 0 & if\ i = j \end{cases}$

The Shortest Path Problem (cont.)

	1	2	3	4	5
1	0	1	∞	1	5
2	9	0	3	2	∞
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	∞	∞	∞	0

W



The Shortest Path Problem (cont.)

	1	2	3	4	5
1	0	1	∞	1	5
2	9	0	3	2	∞
3	∞	∞	0	4	∞
4	∞	∞	2	0	3
5	3	∞	∞	∞	0

W

相鄰矩陣

?

	1	2	3	4	5
1	0	1	3	1	4
2	8	0	3	2	5
3	10	11	0	4	7
4	6	7	2	0	3
5	3	4	6	4	0

D

最短路徑矩陣

The Shortest Path Problem (cont.)

- 分割問題
- 定義 $D^{(k)}[i][j]$ = 從 v_i 到 v_j 其間僅經過 $\{v_1, v_2, v_3, \ldots, v_k\}$ 中之頂點的最短路徑長度

Example

$$D^{(0)}[2][5] = length[v_2, v_5] = \infty.$$

$$D^{(1)}[2][5] = minimum(length[v_2, v_5], length[v_2, v_1, v_5])$$
$$= minimum(\infty, 14) = 14.$$

$D^{(2)}[2][5] = D^{(1)}[2][5] = 14.$ {For any graph these are equal because a}
{shortest path starting at v_2 cannot pass }
{through v_2.}

$D^{(3)}[2][5] = D^{(2)}[2][5] = 14.$ {For this graph these are equal because}
{including v_3 yields no new paths}
{from v_2 to v_5.}

The Shortest Path Problem (cont.)

- 演算法的想法
  - $D^{(0)}[i][j]$代表的是頂點 v_i到v_j 間不能經過其他任何一個頂點所產生最短路徑的長度
    - $D^{(0)} = W$
  - $D^{(n)}[i][j]$所代表的是v_i到v_j間最短路徑長度
    - $D^{(n)}=D$
  - 建立一個遞迴的程序從 $D^{(k-1)}$ 計算出 $D^{(k)}$
  - 由於$D^{(0)}$已知，所以可以從 $D^{(0)}$ 算出 $D^{(n)}$

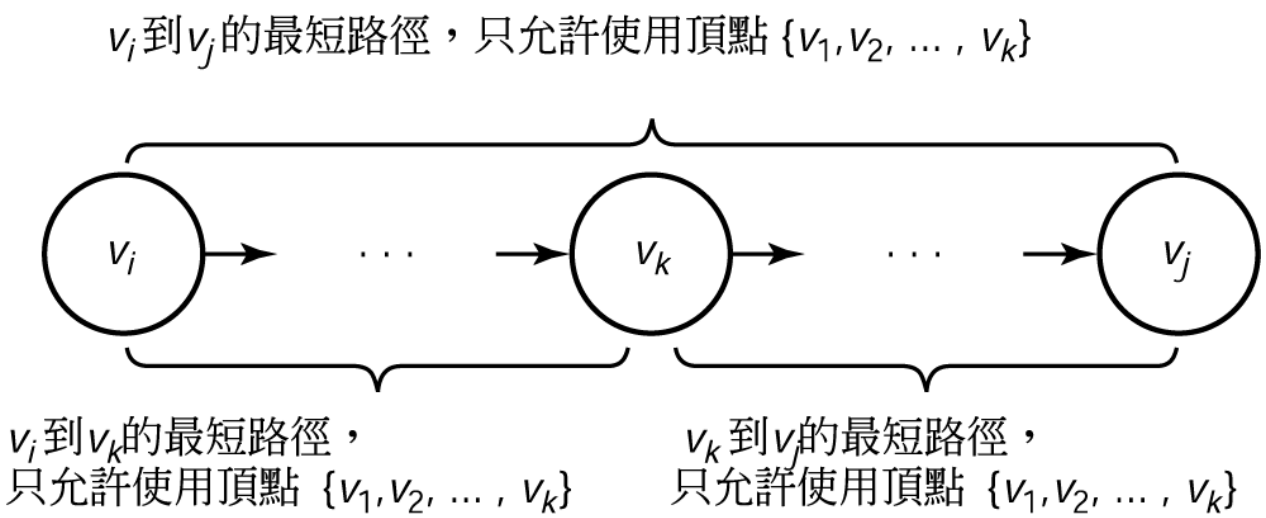
從 $D^{(k-1)}$ 計算出 $D^{(k)}$

- 狀況一
  - 存在一條聯繫 v_i到v_j 同時其間僅經過 $\{v_1, v_2, v_3, \ldots, v_k\}$ 的最短路徑，而該路徑並未經過v_k

$$D^{(k)}[i][j] = D^{(k-1)}[i][j]$$

從 $D^{(k-1)}$ 計算出 $D^{(k)}$ (cont.)

- 狀況二
  - 所有聯繫 v_i到v_j 同時其間僅經過 $\{v_1, v_2, v_3, \ldots, v_k\}$ 的最短路徑，都經過v_k
  - $D^{(k)}[i][j] = D^{(k-1)}[i][k] + D^{(k-1)}[k][j]$



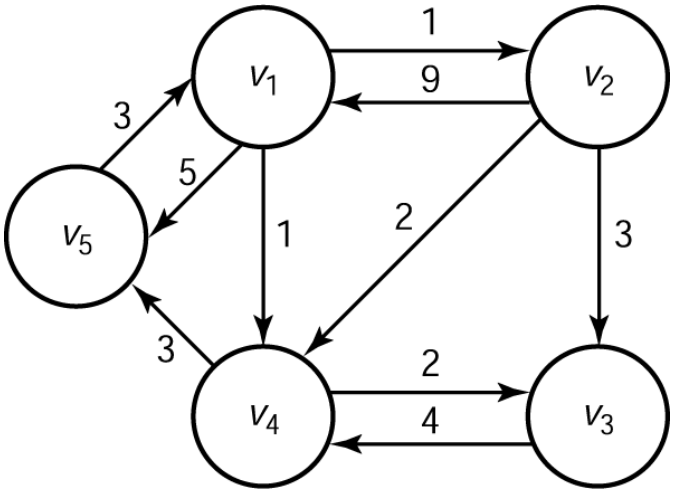
從 $D^{(k-1)}$ 計算出 $D^{(k)}$ (cont.)

- 狀況一範例

$$D^{(5)}[1][3] = D^{(4)}[1][3] = 3$$

- 狀況二範例

$$D^{(2)}[5][3] = 7 = 4 + 3 = D^{(1)}[5][2] + D^{(1)}[2][3]$$



從 $D^{(k-1)}$ 計算出 $D^{(k)}$ (cont.)

- $D^{(k)}[i][j] = minimum(\underbrace{D^{(k-1)}[i][j]}_{\text{狀況一}}, \underbrace{D^{k-1}[i][k] + D^{k-1}[k][j]}_{\text{狀況二}})$
- 範例

$$D^{(1)}[2][4] = minimum(D^{(0)}[2][4], D^{(0)}[2][1] + D^{(0)}[1][4])$$
$$= minimum(2, 9 + 1) = 2$$
$$D^{(1)}[5][2] = minimum(D^{(0)}[5][2], D^{(0)}[5][1] + D^{(0)}[1][2])$$
$$= minimum(\infty, 3 + 1) = 4$$
$$D^{(1)}[5][4] = minimum(D^{(0)}[5][4], D^{(0)}[5][1] + D^{(0)}[1][4])$$
$$= minimum(\infty, 3 + 1) = 4$$
$$D^{(2)}[5][4] = minimum(D^{(1)}[5][4], D^{(1)}[5][2] + D^{(1)}[2][4])$$
$$= minimum(4, 4 + 2) = 4$$

Floyd's Algorithm for Shortest Path

- 問題：計算一張權重圖 (Weighted Graph) 各頂點間的最短路徑(其權重皆為非負值)。
- 輸入：一有向權重圖 (Directed Weighted Graph)，其中共有 n 個頂點，此圖形以相鄰矩陣 W 來表示。
- 輸出：二維陣列 D，其列與行的索引值均由1到n，其中$D[i][j]$即表示第i個頂點到第j個頂點間最短路徑的長度。

Floyd's Algorithm for Shortest Path (cont.)

```
void floyd (int n
            const number W[][],
            number D[][])
{
  index i, j, k;
  D = W;
  for (k = 1; k <= n; k++)
     for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
           D[i] [j] = minimum(D[i][j], D[i][k] + D[k][j]);
```

Time Complexity

- 基本運算：發生在 for-j 迴圈中的指令。
- 輸入大小： n，圖形中頂點的數目。
- for-j 迴圈被包含在迴圈 for-i 之中，然後整個又被包在 for-k 迴圈中，這三個迴圈分別都執行 n 次，所以

$$\mathrm{T}(n) = n \times n \times n = n^3 \in \Theta(n^3)$$

找出最短路徑

- 問題：同演算法3.3，除此之外也要找出最短路徑。
- 額外的輸出：陣列 P ，其列與行的索引值皆為1到 n 。

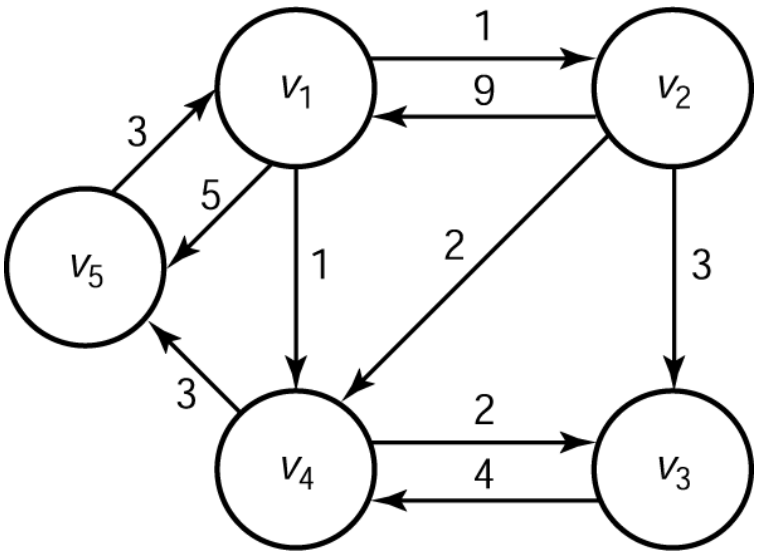
$$P[i][j] = \begin{cases} v_i \text{ 到 } v_j \text{ 的最短路徑上，索引值最大的頂點編號} \\ \text{（若最短路徑上至少有一頂點存在）} \\ 0 \text{（如果 } v_i \text{ 到 } v_j \text{ 的最短路徑上沒有任何頂點存在）} \end{cases}$$

```
void floyd2 (int n,
             const number W[][],
                   number D[][],
                   index  P[][])
{
  index, i, j, k;

  for (i = 1; i <= n; i++)
     for (j = 1; j <= n; j++)
```

```
        P[i] [j] = 0;
  D = W;
  for (k = 1; k <= n; k++)
     for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
           if (D[i][k] + D[k][j] < D[i][j]) {
              P[i][j] = k;
              D[i][j] = D[i][k] + D[k][j];
           }
}
```

	1	2	3	4	5
1	0	0	4	0	4
2	5	0	0	0	4
3	5	5	0	0	4
4	5	5	0	0	0
5	0	1	4	1	0



印出最短路徑

問題：在一權重圖中，印出某頂點到另一頂點最短路徑上的所有頂點。

輸入：由演算法 3.4 所產生的陣列 P，另外再加入兩個索引值 q 和 r，分別 表示圖上兩個頂點的編號。

$$P[i][j] = \begin{cases} v_i \text{ 到 } v_j \text{ 的最短路徑上，索引值最大的頂點編號} \\ \text{（若最短路徑上至少有一頂點存在）} \\ 0 \text{（如果 } v_i \text{ 到 } v_j \text{ 的最短路徑上沒有任何頂點存在）} \end{cases}$$

輸出：v_q 到 v_r 間最短路徑上的所有頂點。

```
void path (index q, r)
{
  if (P[q][r] != 0){
     path (q, P[q][r]);
     cout << "v" << P[q][r];
     path (P[q][r], r);
  }
}
```

動態規劃和最佳化問題

- 建立一個解最佳化問題的演算法步驟如下：
  1. 建立一個遞迴的機制，用它來求取一個問題經過切割後，所產生較小但性質相同問題的解。
  2. 用bottom-up的方式解題，首先由最小的問題開始，逐步向上求取最後整個問題的解。
  3. 用bottom-up的方式建構出最佳解。

最佳化原則 (Principle of Optimality)

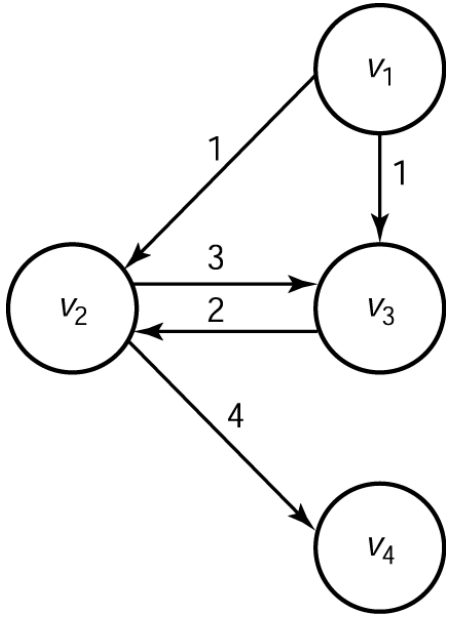
- 並非所有最佳化問題都可以用 Dynamic Programing 解，需符合最佳化原則
- 最佳化原則 (principle of optimality)

一個問題的最佳解同時包含其子問題的最佳解，換句話說，一個問題的最佳解為其子問題之最佳解所組成的

- 範例：如果 v_k 是 v_i 到 v_j 間最短路徑上的點，則該最短路徑中 v_i 到 v_k 以及 v_k 到 v_j 此二子路徑也必定是最短路徑

不符合最佳化原則的反例

- 如何在一個圖形中，找出各個頂點之間「最長」**簡單**路徑。
- 在下圖中，v_1 到v_4 間的最長路徑是$[v_1,v_3,v_2,v_4]$ ，然而其中的一條子路徑 $[v_1,v_3]$，並非 v_1 到 v_3間的最長路徑，因為 $[v_1,v_2,v_3]$ 才是 v_1 到 v_3間的最長路徑 (length$[v_1,v_3]$=1 並且 length$[v_1,v_2,v_3]$=4)
- 不符合最佳化原則



連續矩陣相乘

- 複習一下矩陣乘法是怎麼回事

$$A_{3x2} \cdot B_{2x4} = C_{3x4} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \cdot \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{pmatrix} =$$

$$= \begin{pmatrix} 1\cdot1+2\cdot5 & 1\cdot2+2\cdot6 & 1\cdot3+2\cdot7 & 1\cdot4+2\cdot8 \\ 3\cdot1+4\cdot5 & 3\cdot2+4\cdot6 & 3\cdot3+4\cdot7 & 3\cdot4+4\cdot8 \\ 5\cdot1+6\cdot5 & 5\cdot2+6\cdot6 & 5\cdot3+6\cdot7 & 5\cdot4+6\cdot8 \end{pmatrix} = \begin{pmatrix} 11 & 14 & 17 & 20 \\ 23 & 30 & 37 & 44 \\ 35 & 46 & 57 & 68 \end{pmatrix}$$

連續矩陣相乘 (cont.)

- 看看以下這四個矩陣相乘

A	$\times$	B	$\times$	C	$\times$	D
20×2		2×30		30×12		12×8

- 矩陣相乘的結果與乘法結合順序無關
- 然而，不同的順序需要的乘法次數也不相同

$A(B(CD))$	$30\times 12\times 8$	$+$	$2\times 30\times 8$	$+$	$20\times 2\times 8$	$=$	3,680
$(AB)(CD)$	$20\times 2\times 30$	$+$	$30\times 12\times 8$	$+$	$20\times 30\times 8$	$=$	8,880
$A((BC)D)$	$2\times 30\times 12$	$+$	$2\times 12\times 8$	$+$	$20\times 2\times 8$	$=$	1,232
$((AB)C)D$	$20\times 2\times 30$	$+$	$20\times 30\times 12$	$+$	$20\times 12\times 8$	$=$	10,320
$(A(BC))D$	$2\times 30\times 12$	$+$	$20\times 2\times 12$	$+$	$20\times 12\times 8$	$=$	3,120

連續矩陣相乘 (cont.)

- n 個矩陣相乘的最佳順序 (乘法次數最少) 可以分解成下列其中一種形式

  1. $A_1(A_2 \ldots A_n)$
  2. $(A_1A_2)(A_3 \ldots A_n)$

  …

  k. $(A_1 \ldots A_k)(A_{k+1} \ldots A_n)$

  …

  n-1. $(A_1 \ldots A_{n-1})A_n$

- **這是符合最佳化原則的最佳化問題嗎？**

連續矩陣相乘 (cont.)

給定

$M[i][j]$ = 矩陣 A_i 連乘至 A_j 所需的最少乘法數

$M[i][i] = 0$

d_0 *為* A_1 *的列數* d_k *為* A_k 的行數 $1 \le k \le n$

則 $(A_1 \ldots A_k)(A_{k+1} \ldots A_n)$ 所需的最少乘法總數

$$M[1][k] + M[k+1][n] + d_0 d_k d_n$$

連續矩陣相乘 (cont.)

因此

$$M[1][n] = \underset{1 \le k \le n-1}{minimum}\left(M[1][k] + M[k+1][n] + d_0 d_k d_n\right)$$

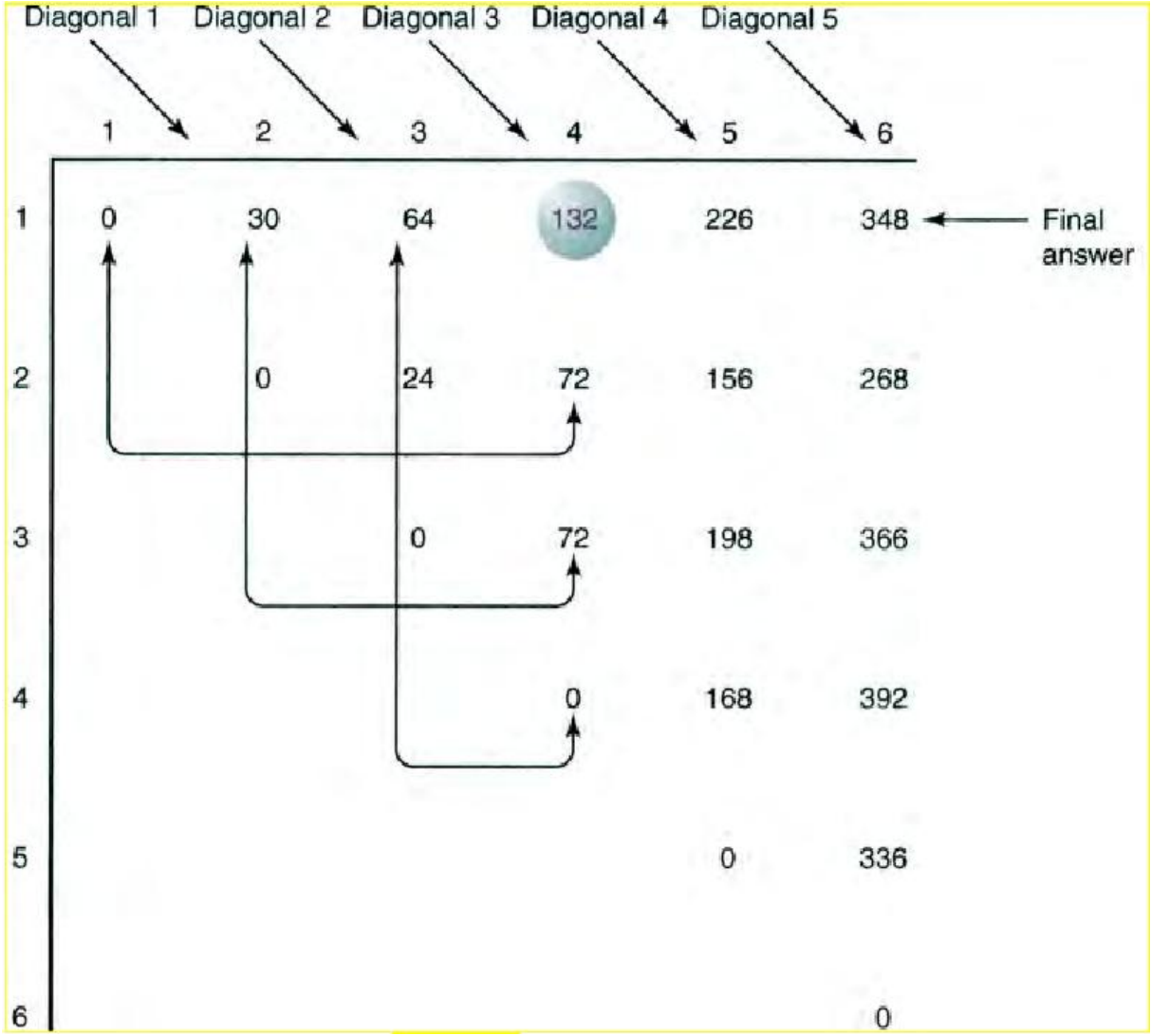
對任意 i, j

$$M[i][j] = \underset{i \le k \le j-1}{minimum}\left(M[i][k] + M[k+1][j] + d_{i-1} d_k d_j\right) \quad \text{if } i < j$$

$$M[i][i] = 0$$

連續矩陣相乘 (cont.)

- M[i][j]的計算牽涉到
  - 同一列位於其左的元素 ($i \le k \le j-1$, M[i][k])
  - 同一欄位於其下的所有元素 ($i \le k \le j-1$, M[$k+1$][j])
  - 利用此性質，我們可以用以下的方式來計算M中的元素：首先，設定主對角線上的值為0；接著我們從主對角線開始，依序由每一條斜線往上計算(對角線1、對角線2....對角線$n-1$)，直到算完對角線 $n-1$ 為止，即可得到M[1][n]



$$
\begin{array}{ccccccccccc}
A_1 & \times & A_2 & \times & A_3 & \times & A_4 & \times & A_5 & \times & A_6 \\
5 \times 2 & & 2 \times 3 & & 3 \times 4 & & 4 \times 6 & & 6 \times 7 & & 7 \times 8 \\
d_0 \quad d_1 & & d_1 \quad d_2 & & d_2 \quad d_3 & & d_3 \quad d_4 & & d_4 \quad d_5 & & d_5 \quad d_6
\end{array}
$$

範例

- 計算對角線0：M[1][1], M[2][2], ..., M[6][6]

  M[i][i] = 0 for $1 \leq i \leq 6$
- 計算對角線1：

$$M[1][2] = \underset{1 \leq k \leq 1}{minimum}(M[1][k] + M[k+1][2] + d_0 d_k d_2)$$
$$= M[1][1] + M[2][2] + d_0 d_1 d_2$$
$$= 0 + 0 + 5 \times 2 \times 3 = 30$$

- 計算對角線2：

$$M[1][3] = \underset{1 \leq k \leq 2}{minimum}(M[1][k] + M[k+1][2] + d_0 d_k d_3)$$
$$= minimum(M[1][1] + M[2][3] + d_0 d_1 d_3,$$
$$M[1][2] + M[3][3] + d_0 d_2 d_3)$$
$$= minimum(0 + 24 + 5 \times 2 \times 4\ ,\ 30 + 0 + 5 \times 3 \times 4) = 64$$

範例 (cont.)

- 計算對角線3：

$$M[1][4] = \underset{1 \le k \le 3}{minimum}(M[1][k] + M[k+1][4] + d_0 d_k d_4)$$

$$= minimum(M[1][1] + M[2][4] + d_0 d_1 d_4\ ,$$
$$M[1][2] + M[3][4] + d_0 d_2 d_4\ ,$$
$$M[1][3] + M[4][4] + d_0 d_3 d_4)$$

$$= minimum(0 + 72 + 5 \times 2 \times 6\ ,\ 30 + 72 + 5 \times 3 \times 6\ ,$$
$$64 + 0 + 5 \times 4 \times 6) = 132$$

- 計算對角線4：
  - 在對角線4上的$M[1][5]$ 、$M[3][6]$ 的計算方式相似
- 計算對角線5：
  - 最終，計算對角線5， 即是矩陣 A_1 乘到 A_6，所需最少的乘法運算次數 $M[1][6] = 348$

連續矩陣相乘最少乘法次數

問題：找出 n 個矩陣相乘所需的最少乘法次數，以及矩陣相乘的順序。

輸入：表示矩陣的數量 n；索引值由 0 到 n 的整數陣列 d，其中 $d[i-1] \times d[i]$ 表示第 i 個矩陣的維度。

輸出：$minmult$ 表示矩陣相乘所需的最少乘法次數；還有一個二維陣列 P，用來儲存矩陣相乘的最佳順序，它的列索引值由 1 到 $n-1$，而行索引值由 1 到 n。$P[i][j]$ 代表矩陣 i 相乘到矩陣 j 最好的分割方式。

```
int minmult (int n,
             const int d[],
             index P [][])
{
  index i, j, k, diagonal;
  int M[1..n] [1..n];

  for (i = 1; i <= n; i++)
     M[i][i] = 0;
  for (diagonal = 1; diagonal <= n - 1; diagonal++)
       // 位在主對角線上方的第一條斜線
```

```
    for (i = 1; i <= n - diagonal; i ++){
        j = i + diagonal;
        M[i][j] =
          minimum (M[i] [k] + M[k + 1][j] + d [i - 1]* d [k] * d [j]);
          i ≤ k ≤ j-1
        P[i][j] = 達到最小乘法次數時的 k 值 ;
    }
  return M[1][n];
}
```

所有情況的時間複雜度（最少乘法次數）

- 基本運算：我們可以用每個不同 k 值所執行的指令作為基本運算，其中包含用來檢查是否為最小值的「比較」指令。
- 輸入大小： n (矩陣的數量)
- 三重迴圈
- 因為$j = i + diagonal$。所以k迴圈的執行次數為
  $j - 1 - i + 1 = i + diagonal - 1 - i + 1 = diagonal$
- for-i 迴圈被執行的次數將會是$n - diagonal$
- 因為$diagonal$的值是由1到$n - 1$ ，所以基本運算的總執行次數為

$$\sum_{diagonal=1}^{n-1} [(n - diagonal) \times diagonal]$$

$$\sum_{d=1}^{n-1}[(n-d)\times d]$$

$$= n\sum_{d=1}^{n-1} d \; - \sum_{d=1}^{n-1} d^2$$

$$= n\cdot\frac{n\,(n-1)}{2} - \frac{n(n-1)(2n-1)}{6}$$

参見 Example A.1, A.2

$$= \frac{n(n-1)(n+1)}{6} \in \Theta(n^3)$$

印出矩陣相乘的最佳順序

- P[2][5] = 4表示矩陣A_2乘到A_5的最佳乘法順序可以分解成

$$(A_2A_3A_4)A_5$$

我們可以藉由拜訪$P[1][n]$找到第一層的分解方式。由於$n = 6$且$P[1,6] = 1$，因此在最佳相乘順序中，第一層的分解方式為

$$A_1(A_2A_3A_4A_5A_6)$$

接下來，我們要找出矩陣 乘到 的最佳乘法順序，所以必需參考 的值。其值為5，所以可以分解成$(A_2A_3A_4A_5)A_6$

而A_2乘到A_5的分解方式仍待求出。接著我們以相同方式查詢$P[2][5]$ 的值，直到所有的分解方式都已決定，最終可以得到答案

$$A_1((((A_2A_3)A_4)A_5)A_6)$$

印出矩陣相乘最佳順序

問題：印出 n 個矩陣相乘的最佳順序。

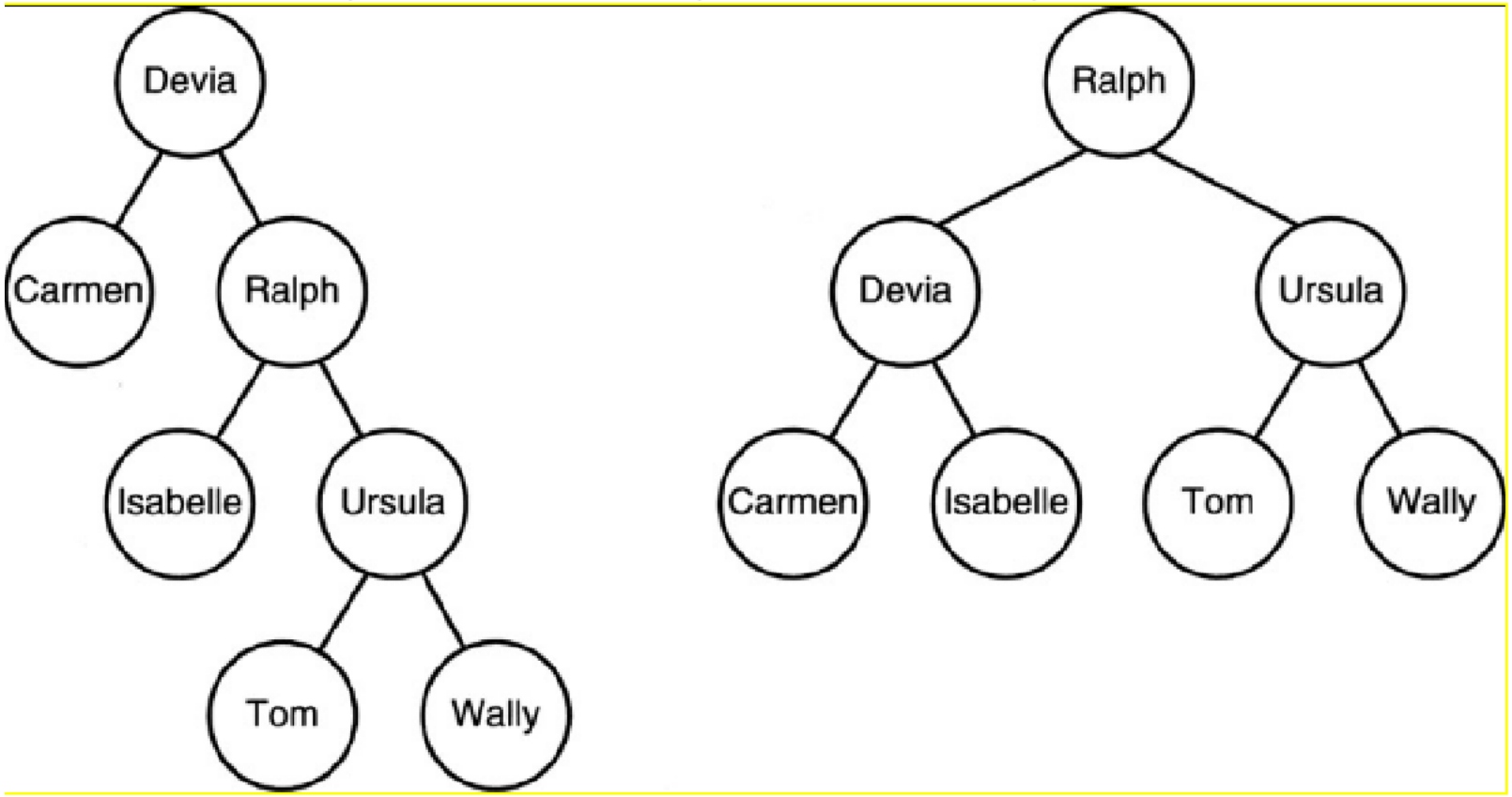
輸入：正整數 n 與由演算法 3.6 所產生的陣列 P，$P[i][j]$ 表示矩陣 i 乘到矩陣 j 的最佳乘法順序中的分割點。

輸出：矩陣相乘的最佳順序。

```
void order (index i, index j)
{
  if (i == j)
     cout << ''A'' << i;
  else {
     k = P[i][j];
     cout << ''('';
     order (i, k);
     order (k + 1, j);
     cout << ''))'';
  }
}
```

最佳二元搜尋樹

- **二元搜尋樹(binary search tree)是一棵由某個有序集合中取出的項(通常稱為key)構成的二元樹。**
  1. 每個節點包含一個key。
  2. 節點 的左子樹中任一節點的key，必需小於等於節點 的key。
  3. 節點 的右子樹中任一節點的key，必需大於或等於節點 的key


Devia
Carmen
Ralph
Isabelle
Ursula
Tom
Wally
Ralph
Devia
Ursula
Carmen
Isabelle
Tom
Wally

二元搜尋樹

問題：在二元搜尋樹中搜尋某個 key(假設此 key 必存在於樹中)。

輸入：*tree* (為一個指標，指向二元搜尋樹)；以及欲搜尋的 key *keyin*。

輸出：一指標 p (指向有包含該 key 的節點)。

```
struct nodetype
{
   keytype key;
   nodetype* left;
   nodetype* right;
};

typedef nodetype* node pointer ;
```

```
void search (node_pointer tree,
             keytype keyin,
             node_pointer& p)
{
  bool found;

  p = tree;
  found = false;
  while (! found)
    if (p->key == keyin)
        found = true;
    else if (keyin < p-> key);
        p = p-> left;                    // 前往左子節點
    else
        p = p-> right;                   // 前往右子節點
}
```

最佳二元搜尋樹

- 在*search*() 程序中搜尋一個key所需之比較(comparison)的次數稱之為搜尋時間(search time)
- 根據各個 key 在二元搜尋樹中的位置 (深度)，搜尋時間會有不

$$depth(key) + 1$$

- 假如各個 key 被搜尋的機率不同，則二元搜尋樹的結構以及各個 key 安排的位置，會影響平均時間複雜度

最佳二元搜尋樹 (cont.)

- 最佳化目標：使整個二元樹的平均搜尋時間是最小的
- 假設 $K_1, K_2, \ldots, K_n$為 n 個key依序排列，並且 p_i為 K_i 被當作search key的機率，而 c_i 是搜尋 K_i 所需比較指令的數目，所以這個樹的平均搜尋時間為

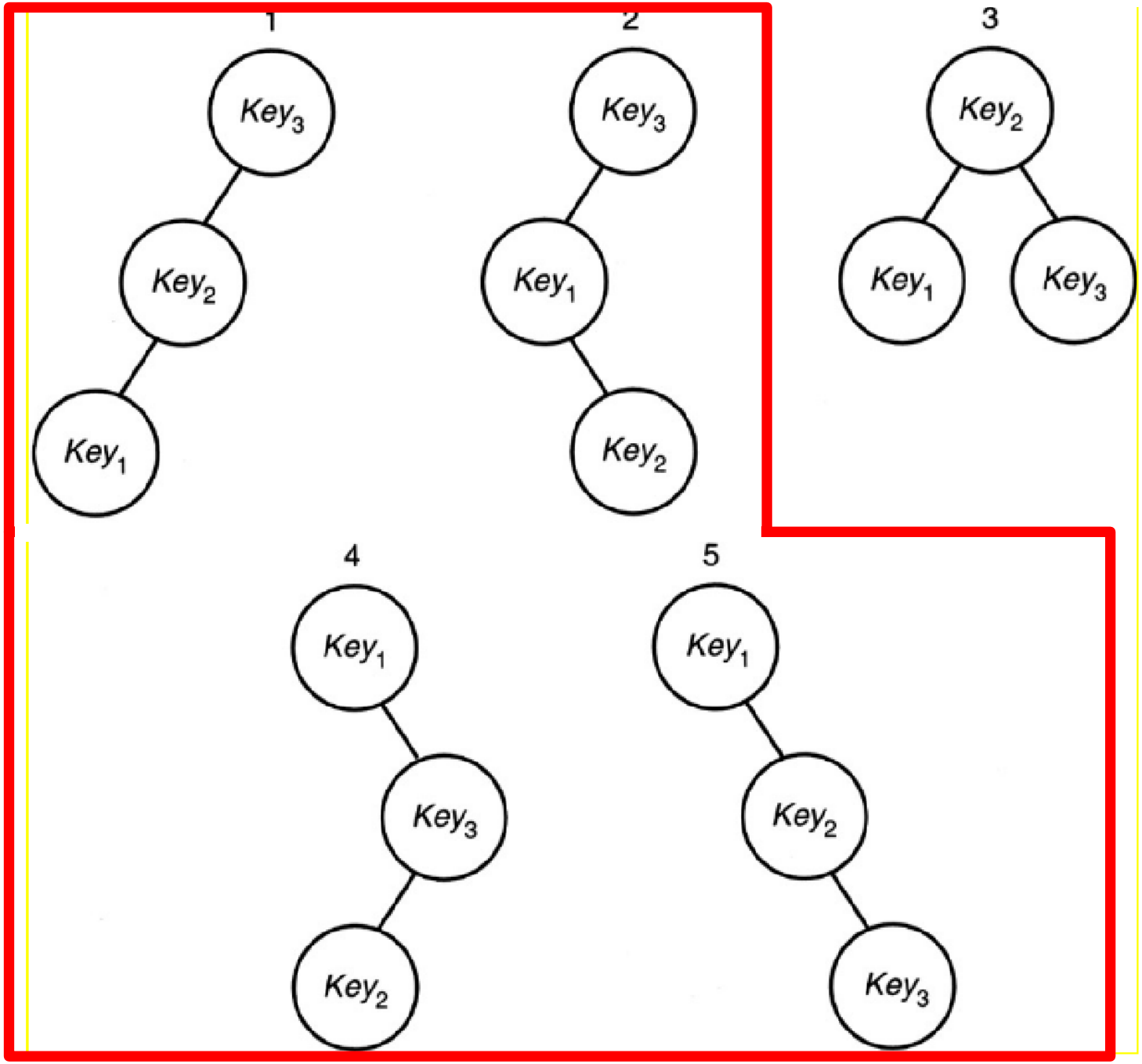
$$\sum_{i=1}^{n} c_i p_i$$

最小化此值

暴力法求解

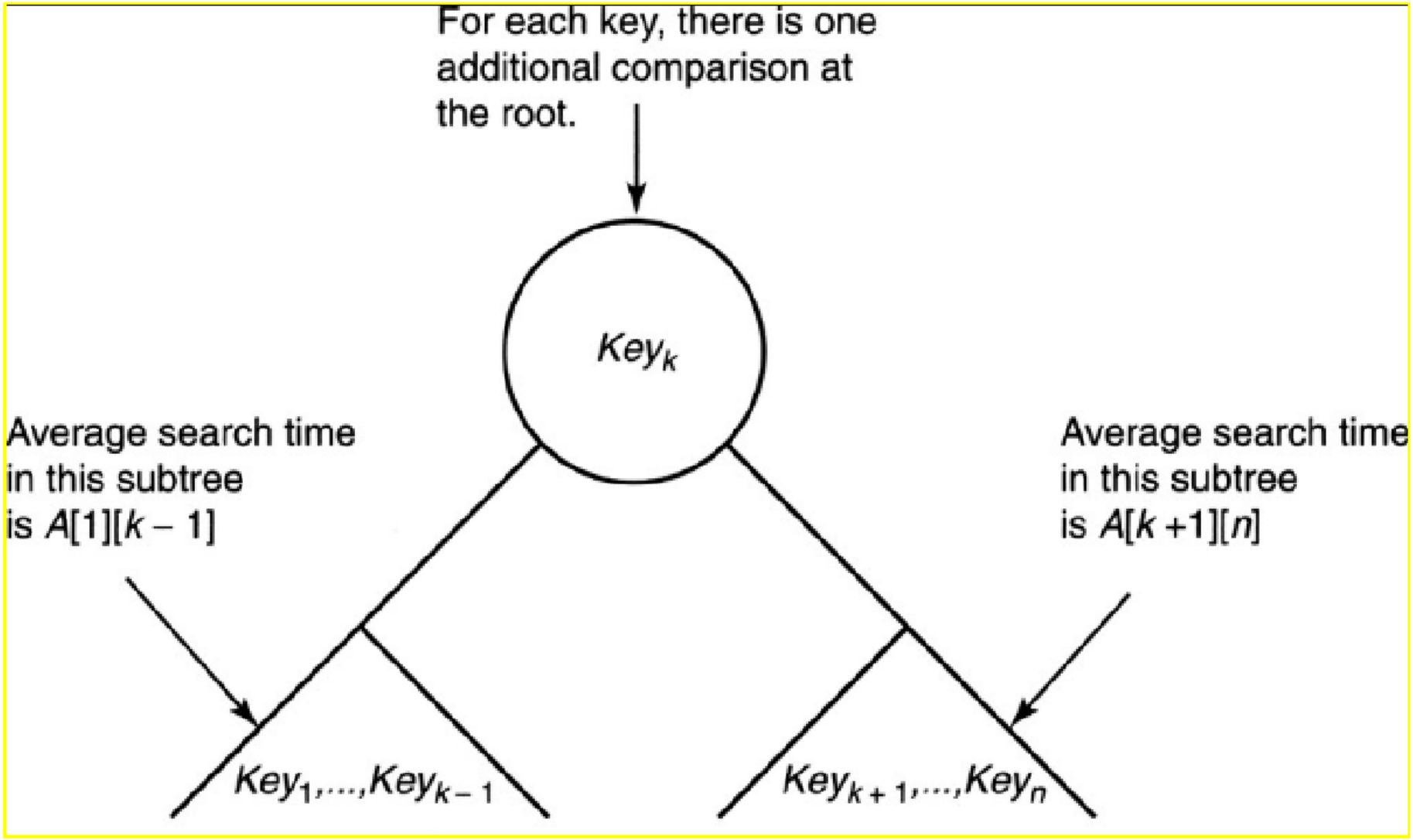
- 列出所有可能的二元搜尋樹，然後再找出其中最佳的一個，必須花費至少 n 的指數時間去計算
- 證明

  考慮 node 數為 n 而深度為 n-1 的二元搜尋樹，在這樣的樹中，除了根節點之外，共有$n-1$層，每一層的節點有兩種選擇，也就是可以做為它的父節點的左子節點或是右子節點。這表示一個二元搜尋樹如果深度為$n-1$ 的話，則共可以組合出2^{n-1}種不同的型式。


1
Key3
Key2
Key1
2
Key3
Key1
Key2
3
Key2
Key1
Key3
4
Key1
Key3
Key2
5
Key1
Key2
Key3

以動態規劃最佳二元搜尋樹

- 假設對於 K_i 到 K_j 存在一顆可以最小化 $\sum_{m=i}^{j} c_m p_m$ 的樹
- 其中 c_m 是在樹中搜尋 key 所需比較指令的數目，我們稱這樣的樹是對 K_i 到 K_j 最佳的樹，並將最佳值以 $A[i][j]$ 來表示
- $A[i][i] = p_i$


For each key, there is one additional comparison at the root.
Key_k
Average search time in this subtree is $A[1][k-1]$
Average search time in this subtree is $A[k+1][n]$
$Key_1,\ldots,Key_{k-1}$
$Key_{k+1},\ldots,Key_n$

以動態規劃最佳化二元搜尋樹 (cont.)

- 最佳樹的任意子樹對這棵子樹上的眾節點們必須是最佳的。因此，此問題便適用前面所提最佳化原則

$$\underbrace{A[1][k-1]}_{\text{左子樹的平均搜尋時間}} + \underbrace{p_1 + \cdots + p_{k-1}}_{\text{左子樹需要的額外比較時間}} + \underbrace{p_k}_{\text{根結點的平均搜尋時間}} + \underbrace{A[k+1][n]}_{\text{右子樹的平均搜尋時間}} + \underbrace{p_{k+1} + \cdots + p_n}_{\text{右子樹需要的額外比較時間}}$$

以動態規劃最佳化二元搜尋樹 (cont.)

上式等於

$$A[1][k-1] + A[k+1][n] + \sum_{m=1}^{n} p_m$$

因為k個樹中必有一個是最佳的，故最佳樹的平均搜尋時間可以寫成以下的式子

$$A[1][n] = \underset{1 \le k \le n}{minimum}(A[1][k-1] + A[k+1][n]) + \sum_{m=1}^{n} p_m$$

我們可以導出以下的式子

$$A[i][j] = \underset{i \le k \le j}{minimum}(A[i][k-1] + A[k+1][j]) + \sum_{m=i}^{j} p_m \qquad i < j$$

$$A[i][i] = p_i \qquad (3.6)$$

$A[i][i-1]$ 及$A[j+1][j]$ 被定義成0

最佳二元搜尋樹 演算法

問題：找出一群 key 的最佳二元搜尋樹，每個 key 都伴隨著一個其做為搜尋鍵的機率。

輸入：n 代表 key 的個數；陣列 p 用來儲存每個 key 被當作搜尋鍵的機率，其中 $p[i]$ 代表 Key_i 的機率。

輸出：變數 $minavg$，代表最佳二元搜尋樹的平均搜尋時間；二維陣列 R，可以藉由 R 建構出最佳樹，它的列索引值由 1 到 $n+1$，行索引值是由 0 到 n，$R[i][j]$ 記錄某一個 key 的索引值，這個 key 被用來當作 Key_i 到 Key_j 所形成最佳樹的根節點。

```
void optsearchtree (int n,
                    const float p[],
                    float& minavg,
                    index R[][])
{
  index i, j, k, diagonal;
  float A[1..n + 1][0..n];
  for (i = 1; i < = n; i++){
     A[i][i - 1] = 0;
     A[i][i] = p[i];
     R[i][i] = i;
     R[i][i - 1] = 0;
  }
  A[n + 1][n] = 0;
  R[n + 1][n] = 0;
  for (diagonal = 1; diagonal <= n - 1; diagonal++)
     for (i = 1; i <= n - diagonal; i ++){      //Diagonal-1 就是位在
                                                // 主對角線上方的第一條線
       j = i + diagonal;
       A[i][j] = minimum(A[i][k - 1] + A[k + 1][j]) +
                 i ≤ k ≤ j
       R[i][j] =  達到最小搜尋次數之 k   ;
     }
  minavg = A[1][n];
}
```

$$\sum_{m=i}^{j} p_m$$

所有情況的時間複雜度(最佳二元搜尋樹)

基本運算：針對每一個 k 值所需執行的指令數目。這其中包含一個用來測試是否為最小值的比較指令。$\sum_{m=i}^{j} p_m$ 這個值並不需要每次都被重新計算。在習題中，你會發現一個更有效率的計算總和方式。

輸入大小：n，key 的數目。

這個演算法的控制部份幾乎和演算法 3.6 一樣。唯一的不同之處在於，當給定 *diagonal* 和 i 值時，基本運算會被執行 *diagonal*+1 次。這個演算法的分析方式和演算法 3.6 相似。

$$T(n) = \frac{n(n-1)(n+4)}{6} \in \Theta(n^3)$$

建立最佳二元搜尋樹

問題：建立最佳二元搜尋樹。

輸入：n 代表 key 的數目；陣列 Key 依序儲存這 n 個 key。陣列 R 是由演算法 3.9 所產生的，其中 $R[i][j]$ 記錄著 Key_i 到 Key_j 所形成最佳樹的根節點之 key 的索引值。

輸出：指標 $tree$，指向包含這 n 個 key 的最佳二元搜尋樹

建立最佳二元搜尋樹 (cont.)

```
node_pointer tree (index i, j)
{
  index k;
  node_pointer p;

  k = R[i][j];
  if (k == 0)
     return NULL;
  else {
     p = new nodetype;
     p-> key = Key[k];
     p-> left = tree(i, k - 1);
     p-> right = tree(k + 1, j);
     return p;
  }
}
```

範例

- 假設我們有下列的陣列 ：

Don	Isabelle	Ralph	Wally
$Key[1]$	$Key[2]$	$Key[3]$	$Key[4]$

- 並且

$$p_1 = \frac{3}{8} \quad p_2 = \frac{3}{8} \quad p_3 = \frac{1}{8} \quad p_4 = \frac{1}{8}$$

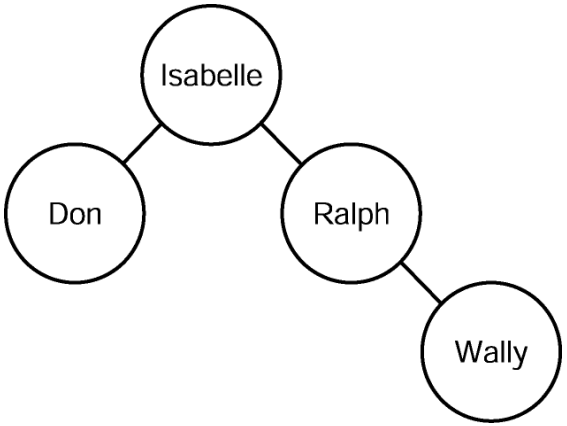
利用前述演算法產生陣列A和R，最小平均搜尋時間為 7/4

	0	1	2	3	4
1	0	$\frac{3}{8}$	$\frac{9}{8}$	$\frac{11}{8}$	$\frac{7}{4}$
2		0	$\frac{3}{8}$	$\frac{5}{8}$	1
3			0	$\frac{1}{8}$	$\frac{3}{8}$
4				0	$\frac{1}{8}$
5					0

A

	0	1	2	3	4
1	0	1	1	2	2
2		0	2	2	2
3			0	3	3
4				0	4
5					0

R



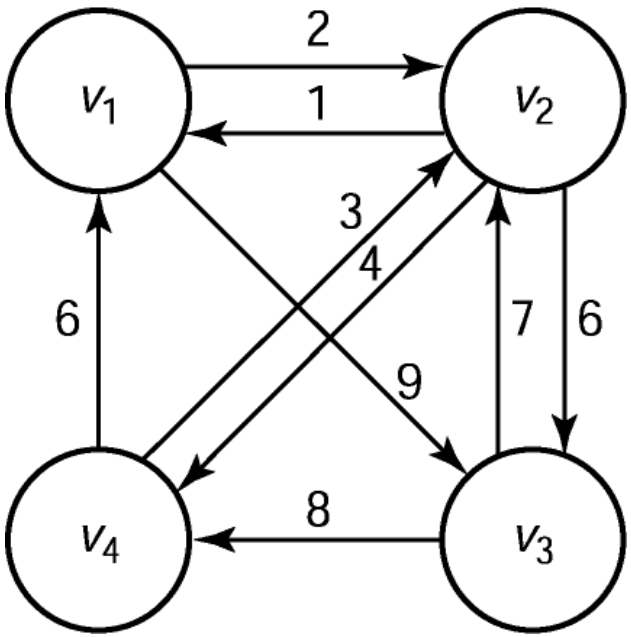
售貨員旅行問題

- 假設有一位售貨員計畫在20個城市間旅行並推銷業務。每個城市間有道路連往其他城市。從售貨員的家鄉城市出發，拜訪了每一個城市一次之後，再回到他的家鄉城市
- 目標: 找出最短的路線

旅行售貨員問題 (cont.)

- 表示成一個權重圖
  - **頂點**代表城市
  - 在有向圖中，我們稱一個**旅程**(tour)為一條路徑，它是由某一個頂點出發，經過所有頂點一次之後，再回到該頂點。我們也把旅程稱為**漢米爾頓迴路** (Hamiltonian circuit)
  - 最佳旅程(optimal tour)是指該路徑擁有最小長度的特性。售貨員旅行問題的解答，就是要找到一個**最佳旅程**

$$length[v_1,\ v_2,\ v_3,\ v_4,\ v_1] = 22$$
$$length[v_1,\ v_3,\ v_2,\ v_4,\ v_1] = 26$$
$$length[v_1,\ v_3,\ v_4,\ v_2,\ v_1] = 21$$



最佳旅程為$[v_1,\ v_3,\ v_4,\ v_2,\ v_1]$

最簡單的想法：找出所有可能的旅程，然後再找出其中路徑長度最小的

在一般情況下，或許每個頂點都有直接的一條邊通往其他頂點，如果我們要找出所有可能的旅程，第二個點有 n-1 種可能，第三個點有n-2種可能，...，第 n 個點只有一種可能。因此，可能的旅程總數為

$$(n-1)(n-2)\cdots 1 \;=\; (n-1)!$$

所花費的時間將超過指數時間

設計動態規劃演算法

- 假設在最佳旅程上，頂點v_k是頂點v_1下一個要拜訪的頂點
- 則旅程上v_k到v_1的這條子路徑也必定是經過所有點至少一次的最短路徑
- 符合最佳化問題原則

設計動態規劃演算法 (cont.)

V = 所有頂點的集合
A = V的一個子集合
W = 相鄰矩陣
$D[v_i][A]$ = 從v_i出發，經過A中的所有頂點一次，再到達v_1的最短路徑長

$$\text{最佳}tour\text{ 長度} = \underset{2 \le j \le n}{minimum}(W[1][j] + D[v_j][V - \{v_1, v_j\}])$$

令 $i \neq 1$且 $v_i \notin A$

$$D[v_i][A] = \underset{j:\, v_j \in A}{minimum}(\, W[i][j] + D[v_j][A - \{v_j\}]\,) \quad \text{if } A \neq \emptyset$$

$$D[v_i][\emptyset] = W[i][1]$$

範例

首先，先考慮空集合的情況：

$D[v_2][\emptyset] = 1, D[v_3][\emptyset] = \infty, D[v_4][\emptyset] = 6$

接下來，考慮所有只包含一個元素的集合：

$$D[v_3][\{v_2\}] = \underset{j:\, v_j \in \{v_2\}}{minimum}(W[3][j] + D[v_j][\{v_2\} - \{v_j\}])$$

$$= W[3][2] + D[v_2][\emptyset] = 7 + 1 = 8$$

同樣地，

$D[v_4][\{v_2\}] = 3 + 1 = 4$

$D[v_2][\{v_3\}] = 6 + \infty = \infty$

$D[v_4][\{v_3\}] = \infty + \infty = \infty$

$D[v_2][\{v_4\}] = 4 + 6 = 10$

$D[v_3][\{v_4\}] = 8 + 6 = 14$

	1	2	3	4
1	0	2	9	∞
2	1	0	6	4
3	∞	7	0	8
4	6	3	∞	0

範例 (cont.)

接下來，考慮所有包含兩個元素的集合：

$$D[v_4][\{v_2, v_3\}] = \underset{j:\ v_j \in \{v_2, v_3\}}{minimum}(W[4][j] + D[v_j][\{v_2, v_3\} - \{v_j\}])$$

$$= minimum(W[4][2] + D[v_2][\{v_3\}]\ ,\ W[4][3] + D[v_3][\{v_2\}])$$

$$= minimum(3 + \infty\ ,\ \infty + 8) = \infty$$

同樣地，

$$D[v_3][\{v_2,\ v_4\}] = minimum(7 + 10\ ,\ 8 + 4) = 12$$

$$D[v_2][\{v_3,\ v_4\}] = minimum(6 + 14\ ,\ 4 + \infty) = 20$$

最後，計算最佳旅程的長度

$$D[v_1][\{v_2, v_3, v_4\}] = \underset{j:\ v_j \in \{v_2, v_3, v_4\}}{minimum}(W[1][j] + D[v_j][\{v_2, v_3, v_4\} - \{v_j\}])$$

$$= minimum(\ W[1][2] + D[v_2][\{v_3, v_4\}]\ ,$$

$$W[1][3]\ +\ D[v_3][\{v_2, v_4\}]\ ,$$

$$W[1][4]\ +\ D[v_4][\{v_2, v_3\}])$$

$$= minimum(2 + 20\ ,\ 9 + 12\ ,\ \infty + \infty) = 21$$

售貨員旅行問題的動態規劃演算法

- 問題：在一個有向權重圖中找出最佳旅程(假設所有的權重都是非負值)。
- 輸入：一個有向權重圖，n代表圖中的頂點數目。此圖用一個二維陣列W表示， W的列與行的索引值都是由1到n。$W[i][j]$ 代表從頂點v_i 到頂點v_j的邊之權重。
- 輸出：
  - 變數$minlength$：最佳旅程的長度
  - 二維陣列P，利用陣列P可以建構出最佳旅程。陣列P的列索引值是由1到n，而行索引值是$V-\{v_1\}$的所有子集合。$P[i][A]$儲存的是由頂點v_i出發，通過集合中所有頂點一次，然後到達頂點 的最短路徑上，位在頂點v_i後的第一個頂點。

```
void travel (int n,
             const number W[][],
             index P [][],
             number& minlength)
{
  index i, j, k;
  number D[1..n][subset of V - {v1}];

  for (i = 2; i <= n; i++)
     D[i][∅] = W[i][1];
  for (k = 1; k <= n - 2; k++)
     for (所有包含 k 個點之 V - {v1} 的子集合 A)
        for (i 使得 i 不等於 1 且 vi 不在 A 中){
           D[i][A] = minimum (W[i][j] + D[j][A - {vj}]);
                     j:vj∈A
           P[i][A] = 造成 minimum 的 j 值;
        }

  D[1][V - {v1}] = minimum (W[1][j] + D[j][V - {v1, vj}]);
                   2 ≤ j ≤ n
  P[1][V - {v1}] = 造成 minimum 的 j 值;
  minlength = D[1][V - {v1}];
}
```

售貨員旅行問題動態規劃演算法之所有情況的時間複雜度

基本運算：主要考慮中間迴圈

輸入大小：n，代表圖中的頂點數目

每個集合 A 包含 k 個頂點，因此需要考慮 n-1-k 個頂點，針對每個頂點，基本運算會被執行k次。因為包含 k 個頂點的$V-\{v_1\}$子集合總共有$\binom{n-1}{k}$個，所以基本運算被執行的總次數為

$$T(n)=\sum_{k=1}^{n-2}(n-1-k)k\binom{n-1}{k}$$

不難看出，

$$(n-1-k)\binom{n-1}{k}=(n-1)\binom{n-2}{k}$$

$$T(n)=(n-1)\sum_{k=1}^{n-2}k\binom{n-2}{k}$$

售貨員旅行問題動態規劃演算法之所有情況的時間複雜度 (cont.)

Theorem 3.1: $\forall\, n \geq 1, \sum_{k=1}^{n} k\binom{n}{k} = n2^{n-1}$

應用定理 3.1

$$T(n) = (n-1)(n-2)2^{n-3} \in \Theta(n^2 2^n)$$

售貨員旅行問題動態規劃演算法之所有情況的空間複雜度

- $M(n)$
  - 計算 $D[v_i][A]$ 及$P[v_i][A]$ 所佔據的空間
  - 因為$V-\{v_1\}$包含$n-1$個頂點而 A 包含 k 個頂點因此 A 有 $\binom{n-1}{k}$種組合
  - 參照 Example A.10 $\sum_{k=0}^{n}\binom{n}{k}=2^n$
  - 因此總體而言，會有2^{n-1}個不同的子集合。
  - 陣列D 和P 的第一個索引值範圍是由1到n，所以

$$M(n)=2\times n2^{n-1}=n2^n\in\Theta(n2^n)$$